Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 5_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
5 2 6 1
Output: 2 1 1 0

Answer

```
#include <stdio.h>

typedef struct {
    int val;
    int idx;
} Pair;

void merge(Pair arr[], int l, int m, int r, int count[]) {
    int n1 = m - l + 1;
    int n2 = r - m;
```

```
    Pair L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    int rightCount = 0;

    while (i < n1 && j < n2) {
        if (L[i].val <= R[j].val) {
            count[L[i].idx] += rightCount;
            arr[k++] = L[i++];
        } else {
            rightCount++;
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        count[L[i].idx] += rightCount;
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

void mergeSort(Pair arr[], int l, int r, int count[]) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m, count);
        mergeSort(arr, m + 1, r, count);
        merge(arr, l, m, r, count);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) scanf("%d", &nums[i]);

    Pair arr[n];
    for (int i = 0; i < n; i++) {
```

```
        arr[i].val = nums[i];
        arr[i].idx = i;
    }

    int count[n];
    for (int i = 0; i < n; i++) count[i] = 0;

    mergeSort(arr, 0, n - 1, count);

    for (int i = 0; i < n; i++) {
        printf("%d ", count[i]);
    }
    printf("\n");
    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula [A * x2 + B * x + C] to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A, B, and C, the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
-1 0 1 2 3 4
-1 2 -1
Output: Array after applying the equation and sorting:
-9 -4 -4 -1 -1 0

Answer

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int A, B, C;
    scanf("%d %d %d", &A, &B, &C);

    for (int i = 0; i < n; i++) {
        arr[i] = A * arr[i] * arr[i] + B * arr[i] + C;
    }

    quickSort(arr, 0, n - 1);

    printf("Array after applying the equation and sorting:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
#include <stdio.h>

void swap(long long *a, long long *b) {
    long long temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(long long arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
```

```
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(long long arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    long long arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }
    int K;
    scanf("%d", &K);

    heapSort(arr, n);

    printf("%lld\n", arr[n - K]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently

determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
17 25 38 47 51 62 79
50
Output: The closest item ID less than or equal to 50 is 47

Answer

```
#include <stdio.h>

int binarySearchClosest(int arr[], int low, int high, int target, int closest) {
    if (low > high) return closest;

    int mid = (low + high) / 2;
```

```
    if (arr[mid] == target) {
        return arr[mid];
    } else if (arr[mid] < target) {
        closest = arr[mid];
        return binarySearchClosest(arr, mid + 1, high, target, closest);
    } else {
        return binarySearchClosest(arr, low, mid - 1, target, closest);
    }
}

int main() {
    int N;
    scanf("%d", &N);
    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    int target;
    scanf("%d", &target);

    int result = binarySearchClosest(arr, 0, N - 1, target, -1);

    if (result == -1) {
        printf("No closest item with an ID less than or equal to %d exists in the
warehouse\n", target);
    } else {
        printf("The closest item ID less than or equal to %d is %d\n", target, result);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2 3 40 50 20
3
Output: Book with ISBN 3 is available at index 1

Answer

```
#include <stdio.h>

int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == x) {
            return mid;
        } else if (arr[mid] < x) {
            low = mid + 1;
```

```
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    int N;
    scanf("%d", &N);
    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    int X;
    scanf("%d", &X);

    int index = binarySearch(arr, N, X);

    if (index != -1) {
        printf("Book with ISBN %d is available at index %d\n", X, index);
    } else {
        printf("Book with ISBN %d is not available in the inventory\n", X);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**